4-3 · MinIO–Iceberg 데이터 파이프라인 구축 · 실습 1.5H

Spark ETL과 MERGE 적재

결정적 중복 제거와 MERGE — 두 번 돌려 검증

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① 중복을 안 줄이고 MERGE 해 MERGE_CARDINALITY 오류를 직접 본다

② 히어독으로 etl_orders.py 를 만들고 키별 최신 한 행만 남긴다

③ 같은 스크립트를 두 번 돌려 행 수가 늘지 않음을 확인한다

④ 새 파일만 추가해 증분 배치를 돌리고 불량 행이 걸러짐을 본다

spark-submit 은 exec -T · --packages 하나와 --conf 네 개 — 앞 챕터와 같습니다

실습 흐름

여섯 단계 — 실패를 먼저, 검증은 두 번

스크립트는 cat > … << 'EOF' 로 만들고, 그 뒤엔 파일만 바꿔 같은 명령을 반복합니다

1. 실패 먼저	2. ETL 작성	3. 실행·확인	4. 멱등성	5·6. 증분·불량
etl_bad.py grep -m1	etl_orders.py row_number	5 · 4 · 4 실측 29초	total rows 4 overwrite 두 줄	08-03 파일 orders_bad

1단계 — 일부러 실패시키기

중복을 안 줄이고 MERGE 하면

1. **cat > scripts/etl_bad.py << 'EOF'**
   row_number 없이 읽기 · 형 변환 뒤 바로 MERGE — 상자째 붙여 넣으면 파일이 생긴다

2. **spark-submit ... etl_bad.py | grep -m1 'MERGE_CARD...'**
   exec -T · --packages · --conf ×4 — 뒤로 종료 로그 수백 줄이라 한 줄만 꺼낸다

3. **[MERGE_CARDINALITY_VIOLATION]**
   대상 한 행(order_id 2)에 원천 두 행이 맞물림 — 나야 정상, 테이블은 4행 그대로

알아 둘 이름

카디널리티 오류

2단계

etl_orders.py — 랜딩 존 → 정제 → MERGE 적재

1) 읽기	2) 형 변환	3) 정제	4) 적재
csv("s3a://raw/orders/") 원천 행 출력	bigint · double to_date	불량 filter row_number = 1	MERGE INTO total rows 출력

cat > scripts/etl_orders.py << 'EOF' 로 만들고, print 세 줄을 grep 으로 골라 본다

「가장 최근 order_date 의 행을 남긴다」는 규칙을 명시 — 2-2 의 ROW_NUMBER 를 PySpark 로 쓰는 것

2단계

핵심 줄 — 정제와 적재

형 변환	cast("bigint") · cast("double") · F.to_date("order_date")
불량 제거	filter("order_id IS NOT NULL AND amount >= 0")
창	Window.partitionBy("order_id").orderBy(F.col("order_date").desc())
최신 한 행	withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
적재	MERGE INTO demo.lakehouse.orders AS t USING updates AS s ON t.order_id = s.order_id

filter 의 두 조건 — 키가 비었거나 금액이 음수인 행은 적재 전에 버린다

3단계

실행하고 출력 세 줄을 읽는다

1. **spark-submit ... etl_orders.py | grep 세 줄**
   exec -T · --packages · --conf ×4 → 원천 행 5 · 정제 후 행 4 · total rows 4(29초)

2. **왜 5 → 4 → 4 인가**
   order_id 2 의 중복 하나가 빠져 4행, 대상도 전부 갱신이라 4행 그대로

3. **spark-sql -e "SELECT * ... ORDER BY order_id;"**
   order_id 2 의 amount 가 35000(8-02) · 1 은 8-01 에만 있어 15000 그대로

4. **UPDATE SET * 는 행 전체를 덮는다**
   1 의 channel 이 NULL → web, 3·4 는 고객까지 8-02 파일 값으로

출력 세 줄

5 · 4 · 4

4단계 — 오늘의 핵심 검증

같은 명령을 한 번 더 — 결과는 같고 기록은 남는다

한 번 더 spark-submit

total rows 가 여전히 4 — 중복 삽입이 없다

- grep -E "total rows" 로 한 줄만
- 2-5 의 UNIQUE 제약이 한 일을
- 여기선 MERGE 의 키 조건이 한다

VS

snapshots 메타테이블

셸에서 spark-sql -e 로 이력 조회

- 끝에 overwrite 두 줄 — 첫 실행 · 재실행
- MERGE 는 파일을 통째로 다시 써 overwrite
- 실패한 etl_bad 는 줄을 남기지 않는다

5·6단계

파일만 추가하고 같은 스크립트 — 실무의 하루

올린 파일	원천 행	정제 후	total rows	무슨 일이
첫 실행(08-01·02)	5	4	4	2 갱신 35000 · 신규 없음
같은 명령 재실행	5	4	4	변화 없음 · overwrite 한 줄
orders_2026-08-03	7	5	5	4 갱신 61000 · 5 신규
orders_bad	10	6	6	6(음수)·빈 키 걸러짐 · 7 신규

매번 랜딩 존 전체를 읽는 방식 — 파일이 수천 개면 느려진다. 날짜 경로(dt=...)는 8주차 Airflow 에서

스크립트는 한 글자도 안 고쳤다 · 정리: mc rm 으로 orders_bad.csv 를 빼면 랜딩 존 파일 셋

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. etl_bad.py 에서 MERGE_CARDINALITY_VIOLATION 을 봤다
2. etl_orders.py 첫 실행 출력이 원천 5 · 정제 4 · total 4 다
3. 2 는 35000, 1 의 channel 은 web 으로 채워졌다
4. 재실행해도 total rows 4, snapshots 끝에 overwrite 두 줄
5. 08-03 · bad 뒤 orders 6행(1~5, 7), 랜딩 존 파일 셋

4-3 · 실습 1.5H 마무리

다음은 정리·회고 (0.5H)

오늘 만든 파이프라인 다섯 단계를 한 장으로 정리하고, 4-4 Trino 가 읽을 테이블 상태를 확인합니다

- ✓ **랜딩 → 읽기 → 형 변환 → 정제 → 적재**
- ✓ **체크리스트 · 자주 겪는 오류**
- ✓ **orders 행 수 적어 두기**
- ✓ **4-4 Trino 준비물**

가져갈 것

orders 6행(1~5, 7) · 불량 파일 삭제